# Rigorous Evaluation of LLM Components in HPC Research

**Daniel Nichols*, Abhinav Bhatele**, Harshitha Menon***

*Lawrence Livermore National Laboratory **University of Maryland

This work is performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

# Section 1: Introduction

- Tutorial Objectives
  - Learn how to properly evaluate LLMs in HPC and scientific research
  - Learn the dangers and common pitfalls from using LLMs in research

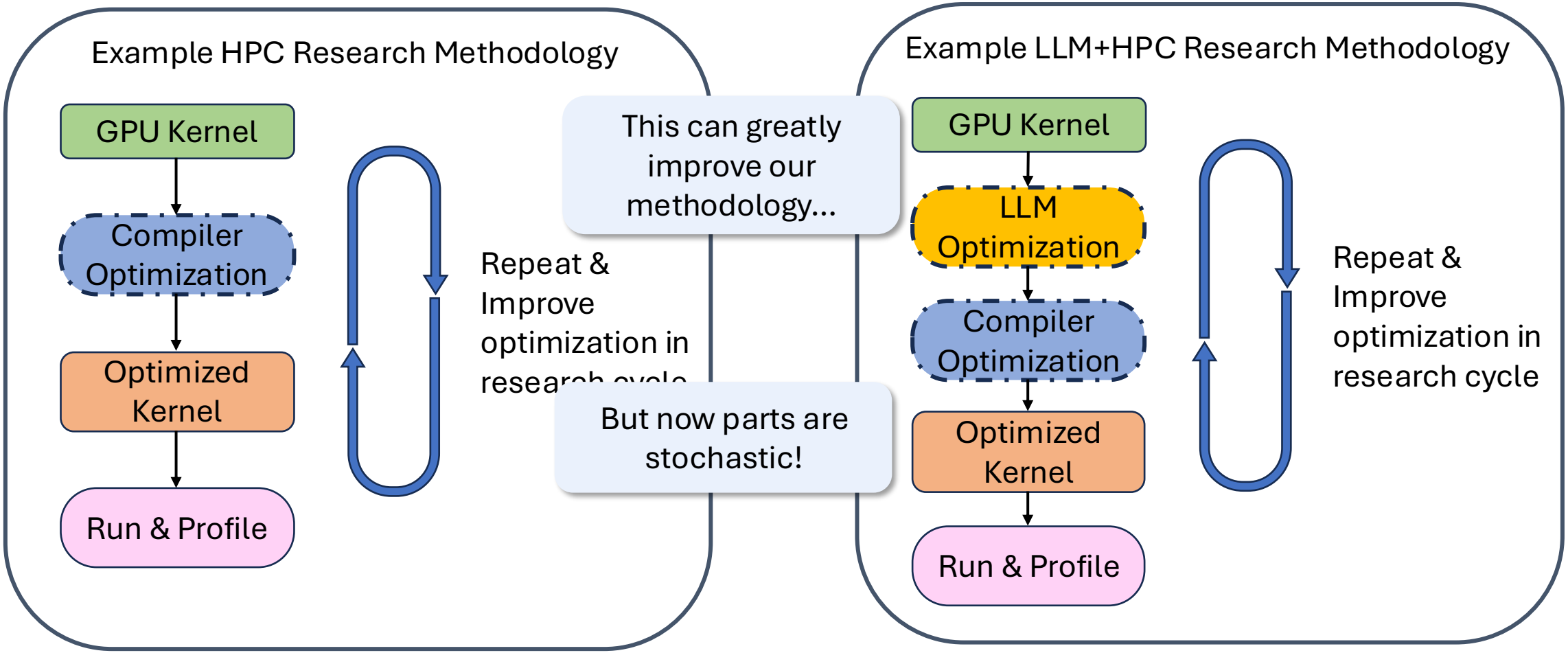
- Overview
  - Intro to LLMs
  - How to Lie with LLM Results
  - Experimental Design for Stochastic Components
  - Transparency and Reproducibility
  - Human Validation: When and How
  - Baselines, Metrics, and Ablations
  - HPC and Scientific Computing Specifics

# Tutorial site



https://parallelcodefoundry.github.io/tutorials/isc-2026-llm-eval/

# Why this tutorial?



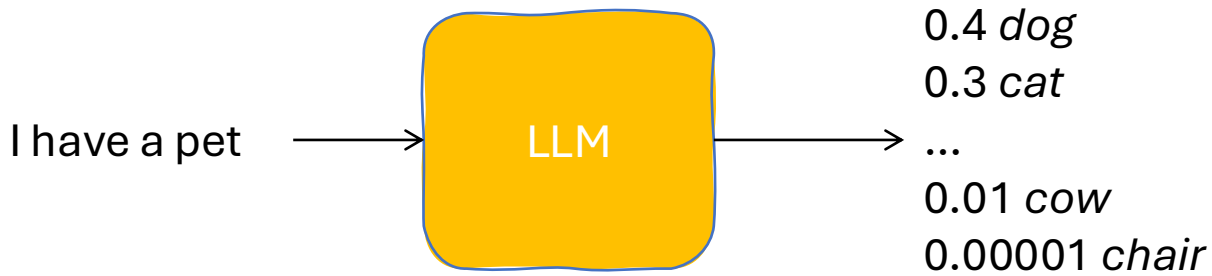
# Section II: Introduction to LLMs

- Goal for this section: shared vocabulary for using LLMs
- You should be able to answer:
  - What are tokens?
  - What is text generation?
  - Where does randomness come from?
  - What is a model?

- Overview
  - What is an LLM?
  - What are tokens?
  - Text Generation
  - Sampling: How Randomness is Introduced
  - Randomness in LLMs

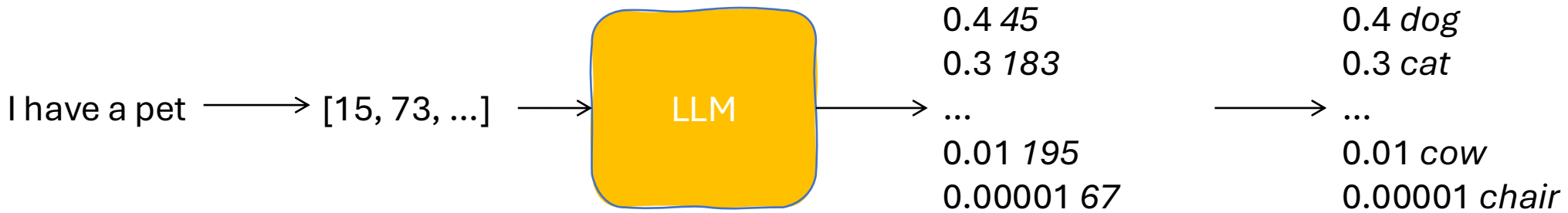
# What is an LLM?

- Large Language Models are neural networks based on the transformer architecture that are state-of-the-art for modeling sequential data (e.g. text, language)
- Often trained for causal language modeling
  - Input: text sequence → output: probabilities of next token



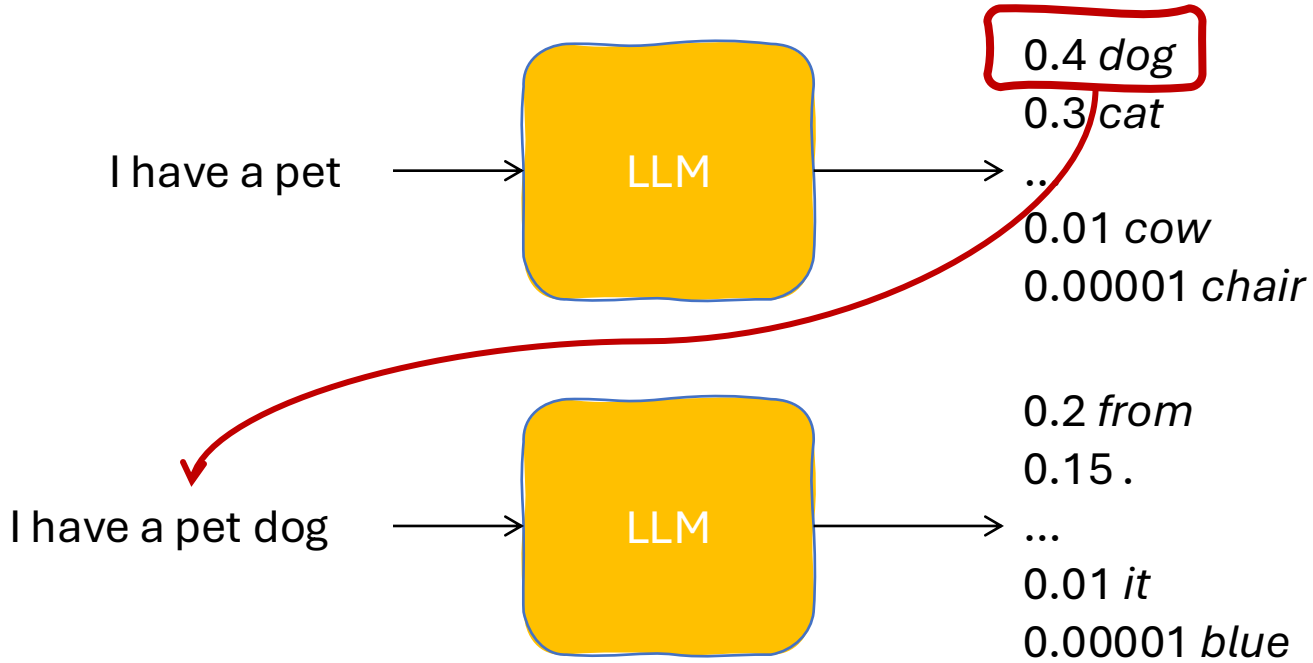
# What are tokens?

- LLMs learn from integer sequences, not text
- *Tokenization* is the process of mapping text to integer sequences
- LLMs take sequences of tokens as input and outputs token probabilities
- Tokens *do not* correspond 1-1 with words or characters



# Text Generation

- Auto-regressive text generation is the process of repeatedly sampling the next token to generate sequences



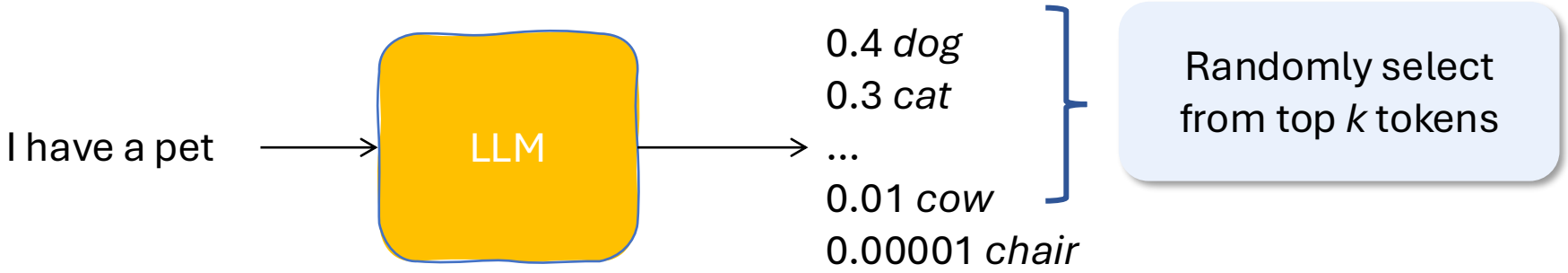
# Text Generation

- Auto-regressive text generation is the process of repeatedly sampling the next token to generate sequences
- Stop when the model generates stop or EOS token
- Note: early differences in sequences can cascade into very different outputs

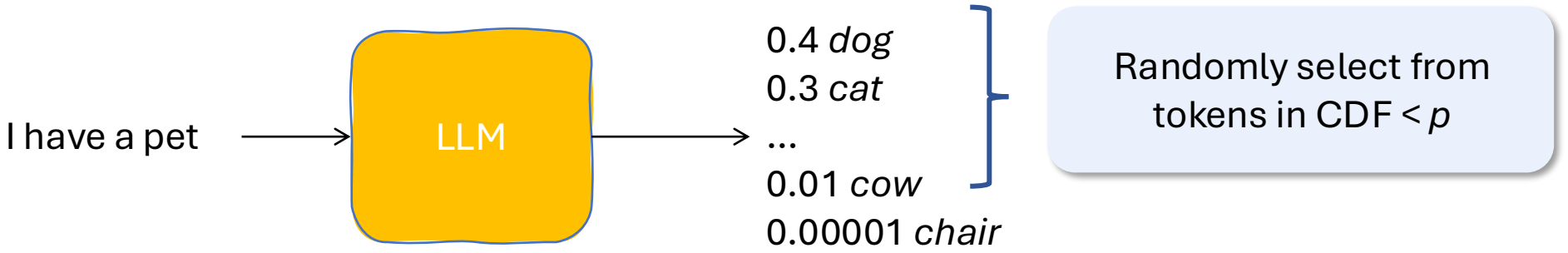
# Sampling: The Introduction of Randomness

- Using the most probable next token leads to bad, repetitive generations (A. Holtzman et al. “The Curious Case of Neural Text Degeneration”)
- Randomly sampling yields much better results
- Popular sampling strategies
  - Top-K
  - Top-P or Nucleus Sampling
  - Beam search
- Temperature often used to scale probabilities before sampling

# Top-K Sampling



# Top-P Sampling



# Randomness in LLMs

- Transformers with fixed weights and inputs are deterministic in the log-probs they output *, **
- Randomness is introduced by:
  - Prompt variance: multiple ways of prompting the same question, e.g. “Find the square root of N” vs “What multiplied by itself equals N?”
  - Sampling: Using randomness to sample the next token in text generation

* Parallelism, floating point arithmetic, and MoEs can cause non-determinism in some implementations
** Commercial/closed models may change system prompts and/or weights without notice

# Demo 1: LLM Randomness

- See demo 1 to try different sampling strategies and their impact on generation outcomes



# Section III: How to Lie with LLM Results

- Goal for this section: understand where LLMs can go wrong when used in research components

- Overview
  - Lie 1: Not considering randomness
  - Lie 2: Cherry picking examples
  - Lie 3: Not considering other axes of evaluation
  - Lie 4: Benchmark leakage / data contamination
  - Lie 5: LLM-as-a-judge, rubric drift
  - Lie 6: Human-as-a-judge, inconsistent evaluation
  - Lie 7: Unreported human intervention
  - Lie 8: Not reporting cost / GPU hours
  - Lie 9: Conflating Formatting Abilities with Technical Abilities

# Lie #1: Not Considering Randomness

- LLM outputs are *random*; reporting metrics for a single output can be misleading
- What should be done
  - You should report: *What is the probability/likelihood that a model outputs the correct solution?*
  - Many works/benchmarks employ this through probabilistic metrics e.g. mean accuracy, pass@k
- Often only sampling variance is covered
  - Rarely we're benchmarking LLMs on prompts, but rather *problems*
  - Prompt variance matters! (Y. Zi et al. "More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation")

# Lie #2: Cherry Picking Examples

- Another facet of random outputs from LLMs is that we must report all results
- Only reporting the “best” results can mislead to thinking the LLM is doing better than it actually is
- Sampling parameters need to be transparent: If the LLM tool only works for some sampling parameters, this is *ok*, but it needs to be reported

# Lie #3: Ignoring Important Axes of Evaluation

- To avoid lies 1-2 practitioners often sample many outputs and, thus, need easy-to-automatically-compute metrics for evaluation
- This often leads to Lie #3: using too simple of metrics that don't paint the whole picture
- Consider evaluating LLMs on generating MPI code

This is functionally correct, but not really correct

```
def compute_and_share(data):
    output = compute(data)
    for rank in other_ranks:
        mpi_send(output, rank)
```

This is what we want to see the LLM generate

```
def compute_and_share(data):
    output = compute(data)
    mpi_bcast(output)
```

# Lie #4: Benchmark Leakage and Data Contamination

- If your benchmark is in the training set, then the model could be over-fitting the answers
- You're testing how well the model can memorize your problems and not actually solve them
- This is only okay if your benchmark is *exhaustive*, which is almost never the case
- Even if your questions are private, make sure they are not too similar to existing public sources

# Lie #5: LLM-as-a-Judge, Rubric Drift

- Rubric drift: small changes in prompts and judge models can drastically alter results
- LLM judging often conflates style and correctness
- LLM-based evaluation is a useful new research tool, but is *insufficient* and *potentially dangerous* for scientific evaluation

# Lie #6: Human-as-a-Judge, Inconsistent Eval

- Humans are just as prone to evaluation biases as LLMs
- Evaluation criteria not public or precise. Evaluation data not public
- Social science fields have best practices for using humans to evaluate / rank / label data. It is still very difficult to implement correctly.
- As with lie #5, human judgement is *insufficient* and *dangerous* in scientific results reporting

# Lie #7: Unreported Human Intervention

- LLM / Agent pipeline relied on human intervention somewhere and it is not reported
  - Data cleaning between components
  - Prompt design and/or variation
  - Sampling parameter and/or model choice changes
- Approach is not reproducible without documenting and studying these human interventions

# Lie #8: Not Reporting Cost / GPU Hours

- API costs or GPU time when self-hosting are imperative to report, particularly when comparing with other approaches
- If an approach yields better results, but costs significantly more this is of interest to users of the approach
- All costs and GPU hours need to be reported
  - Costs fluctuate, so token counts should also be reported.

# Lie #9: Conflating Formatting Abilities with Technical Abilities

- If we are testing an LLM/Agent on its ability to do task *X*, then we don't want to conflate performance on *X* with performance on formatting outputs for task *X*.
- An example: You task an LLM with solving a math problem and outputting the final number; your script checks accuracy with `int(llm_answer) == ground_truth`.
  - The LLM would get this wrong if it generated "The answer is …"
  - Even if it knew the answer we've marked it as wrong
- Give LLMs the benefit of the doubt in parsing outputs, unless your task *X* includes formatting as part of the task

# Section IV: Experimental Design for Stochastic Components

- Goal for this section: how to design experiments that rely on stochastic subcomponents

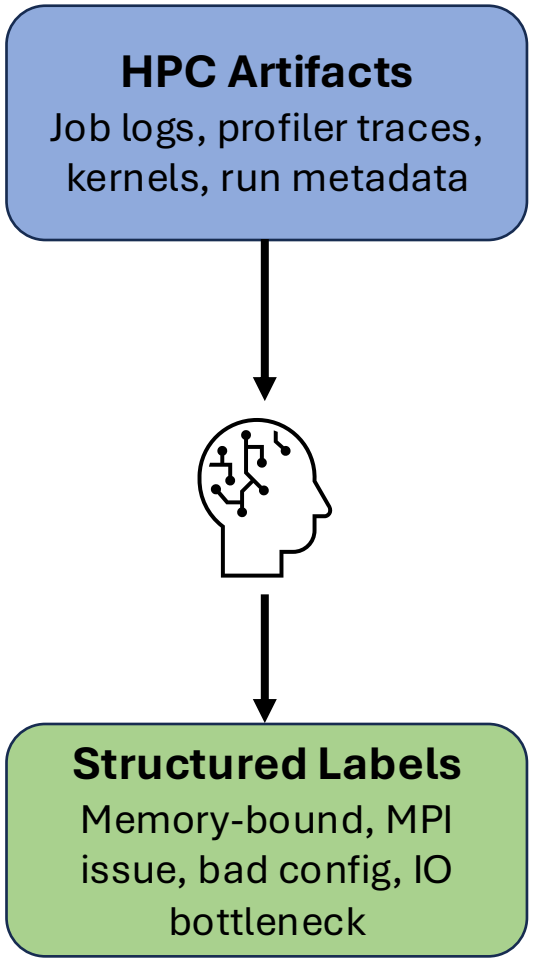
- Overview
  - How are LLMs used in experiments
  - Why careful consideration is important
  - Structured inputs/outputs; mitigating stochasticity
  - Testing individual subcomponents
  - Ablation studies on system
  - Human validation

# How are LLMs used in experiments

- Annotation
- Judge
- Synthesis
- Generation
- Tools
- Benchmarking
- Subject

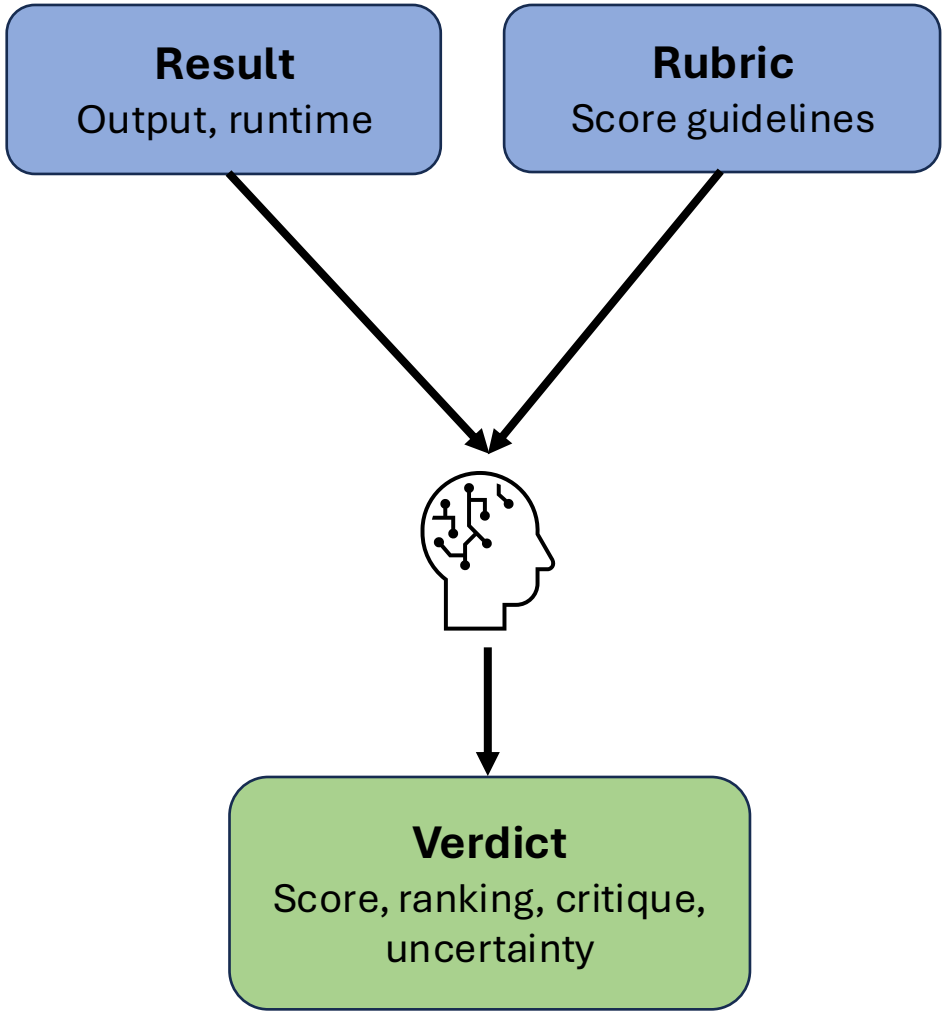
# How are LLMs used in experiments: Annotation

- LLMs label research artifacts such as code, performance data, parallelism type, etc.
- Can apply a predefined coding guide or suggest candidate codes.



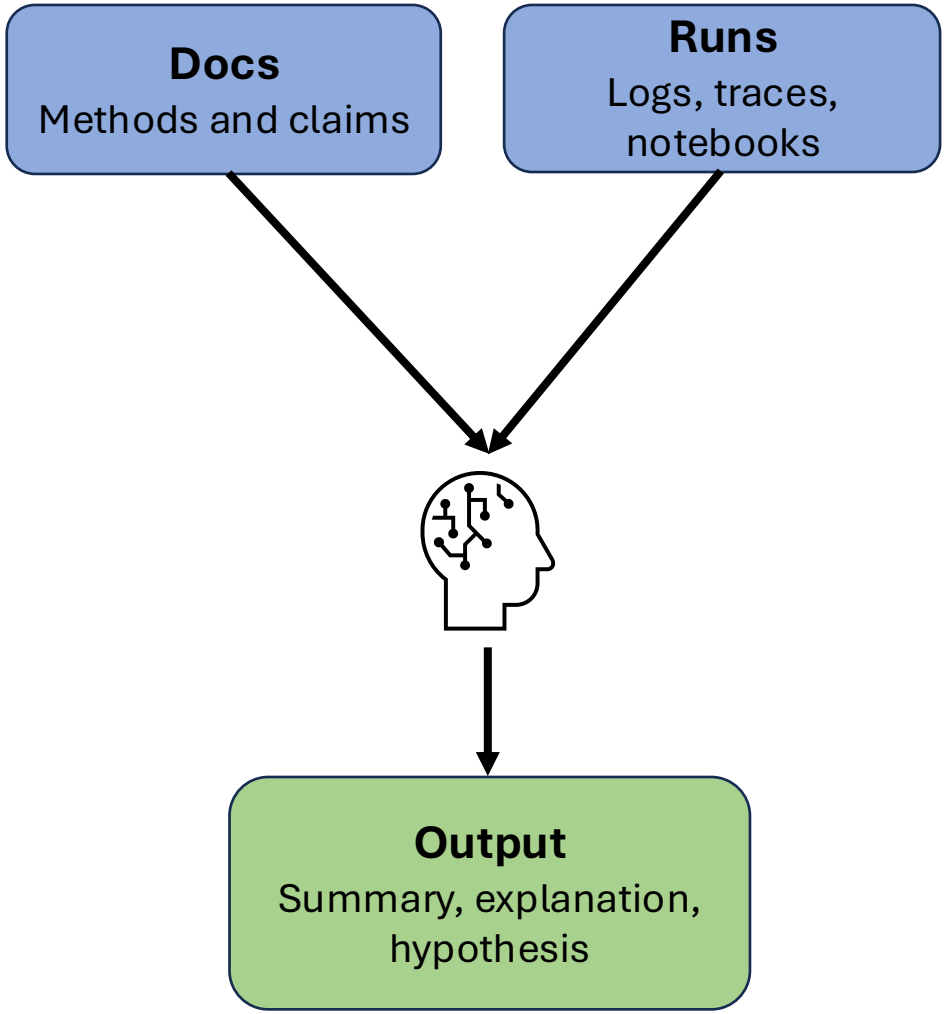
# How are LLMs used in experiments: Judge

- LLMs score, rank, or compare HPC artifacts against a rubric, such as whether an optimization is plausible, a generated explanation is correct, or a run result supports a claim.



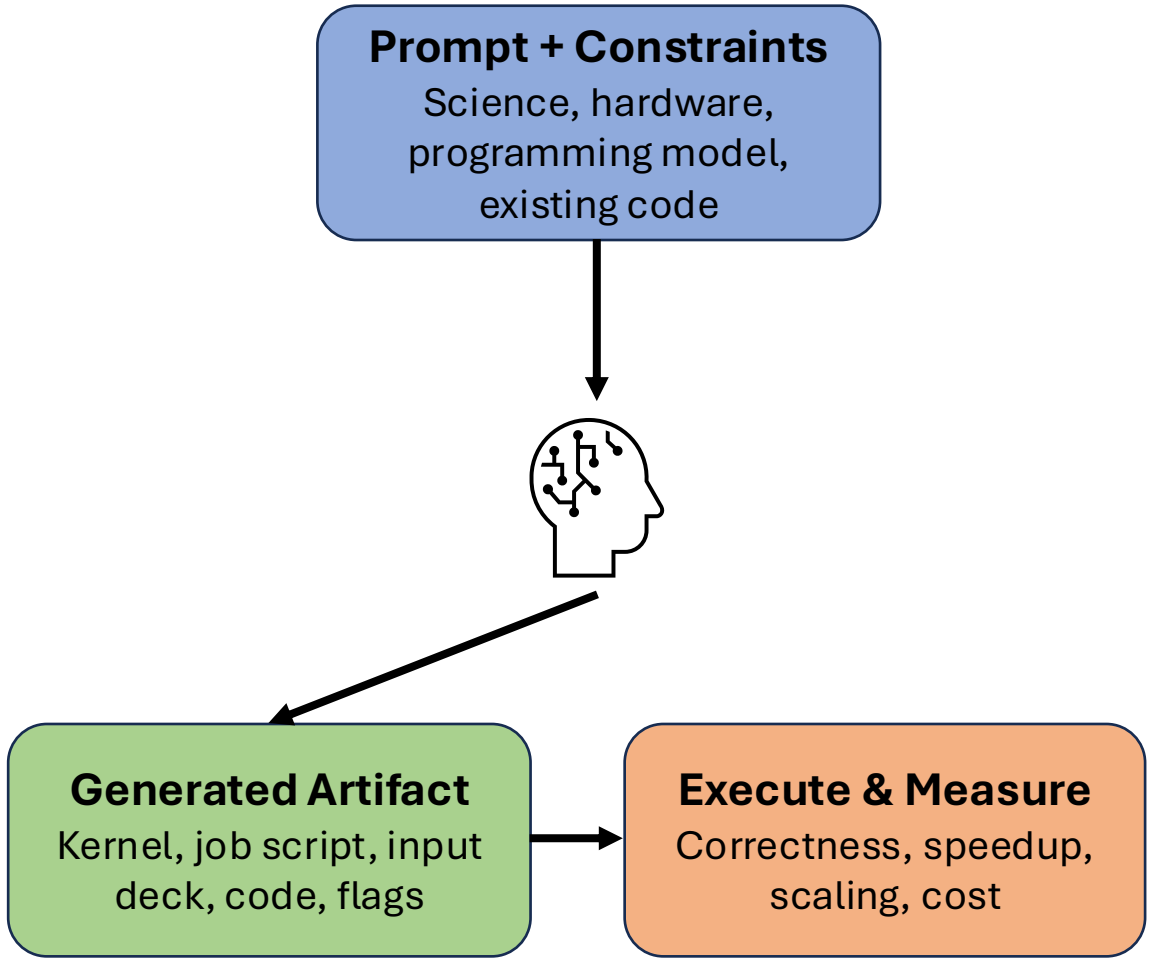
# How are LLMs used in experiments: Synthesis

- LLMs combine evidence across papers, documentation, job outputs, profiler reports, and experiment notes to produce summaries or hypotheses.
- Generate synthetic kernels, input decks, configurations, or expected failure cases for later testing and evaluation.



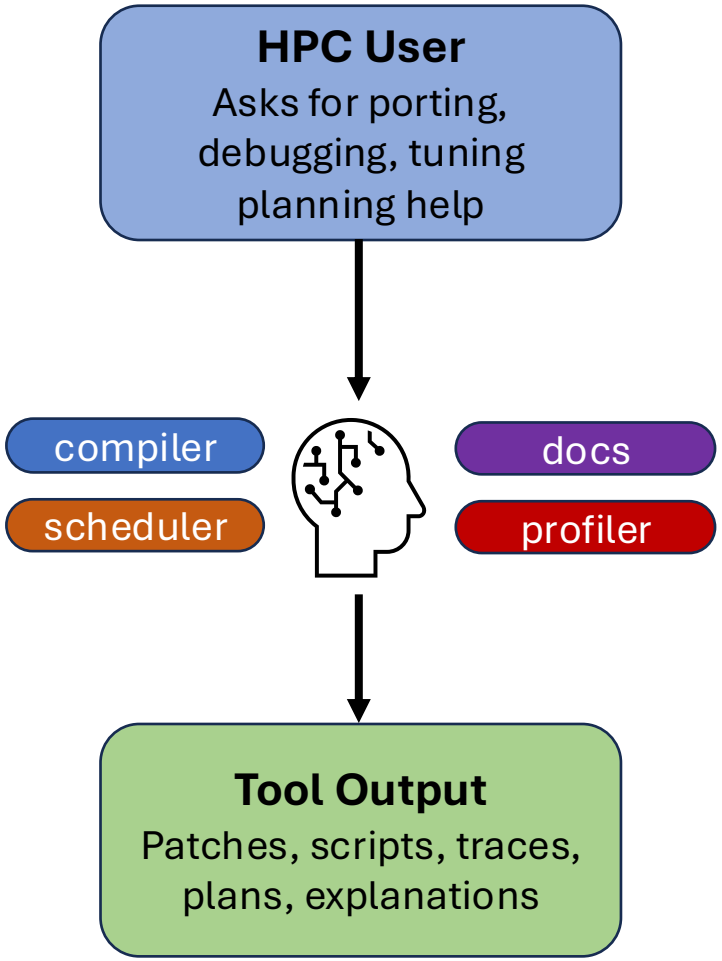
# How are LLMs used in experiments: Generation

- Generate or rewrite kernels, MPI/OpenMP/CUDA code, job scripts, input decks, compiler flags, launch parameters, or workflow configurations.



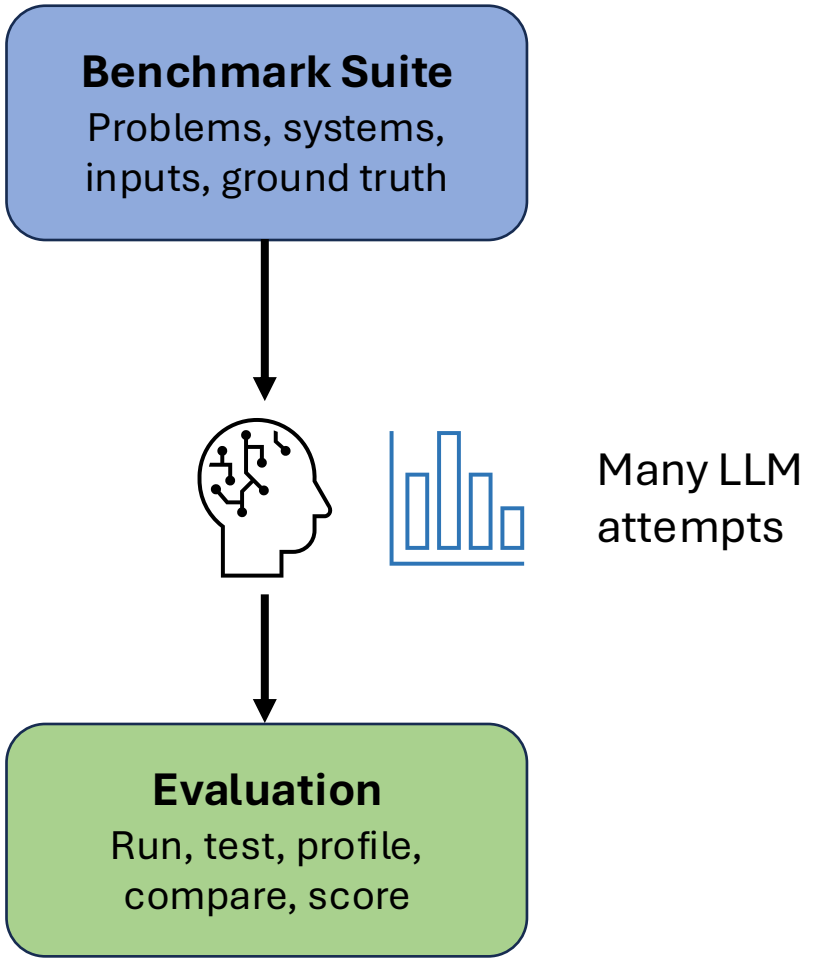
# How are LLMs used in experiments: Tools

- LLMs are embedded into HPC tools or agents for tasks such as code porting, performance debugging, job-script generation, experiment planning, or workflow automation.



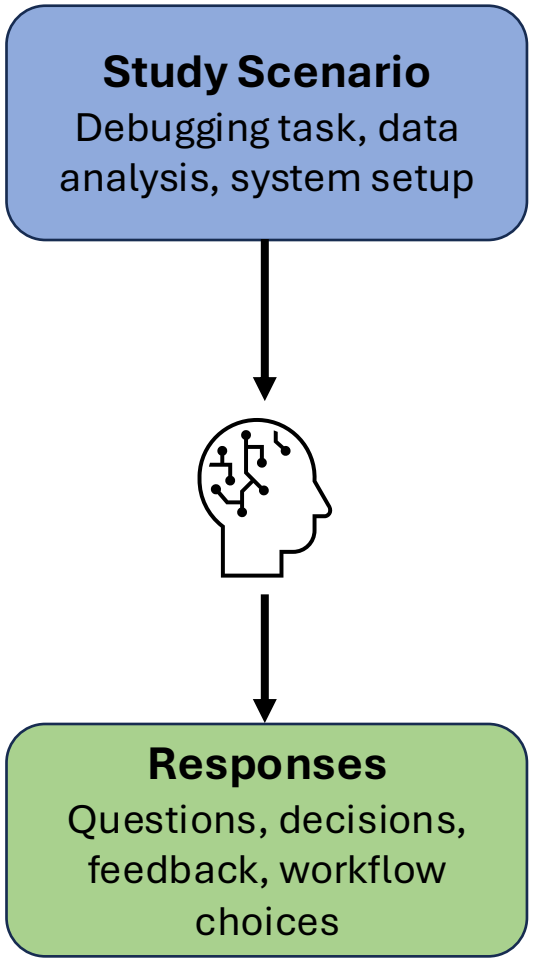
# How are LLMs used in experiments: Benchmarking

- Benchmarks measure LLM performance on standardized HPC tasks with reference problems, datasets, systems, and metrics.



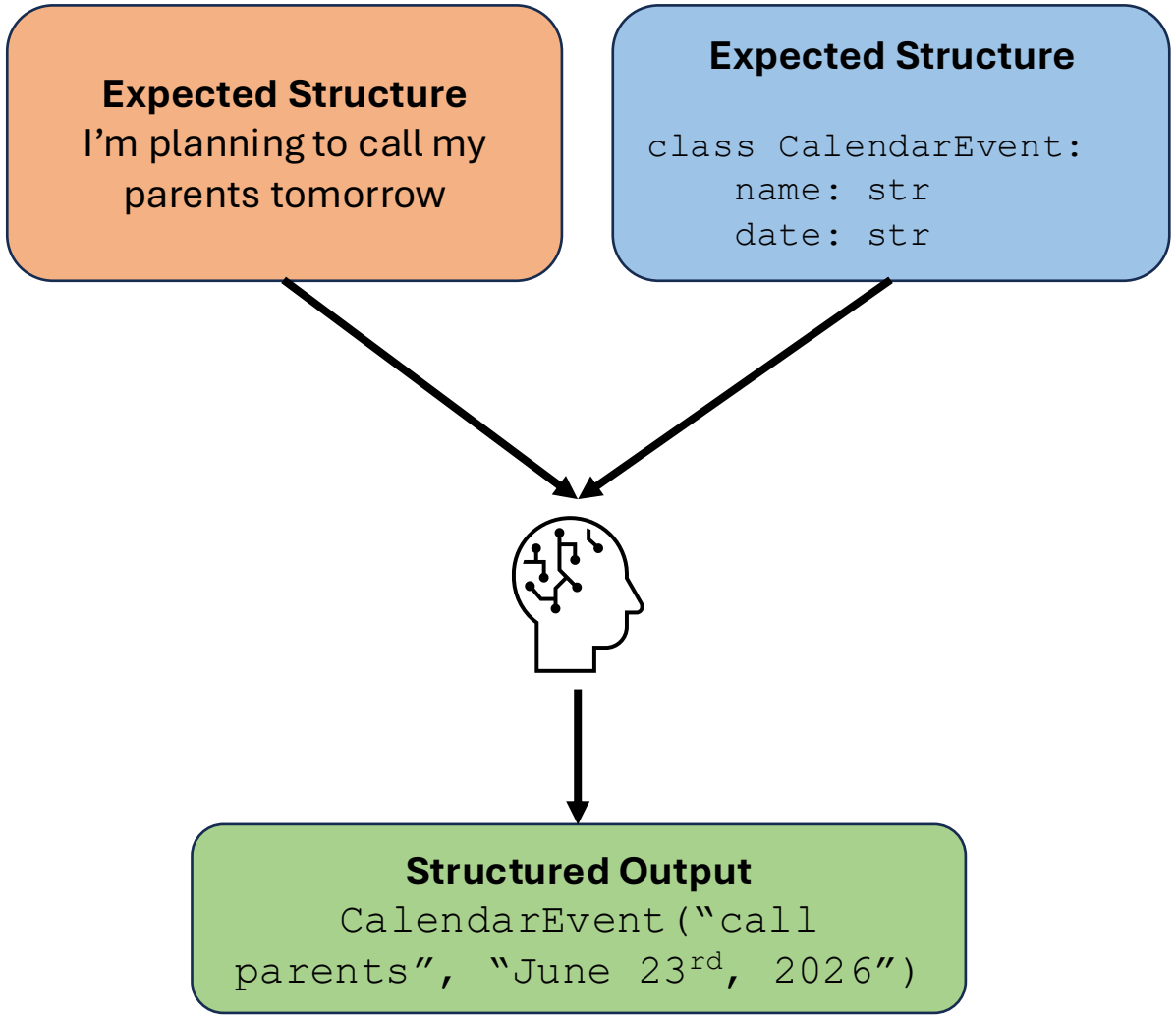
# How are LLMs used in experiments: Subjects

- LLMs act as simulated HPC users, domain scientists, performance engineers, or system operators in studies of workflows and interfaces.



# Structured Inputs/Outputs: Mitigating Stochasticity

- Most LLM/Agent frameworks support structured outputs
- Force the LLM to output JSON with particular fields
- Errors or retries if invalid structure
- Reduces potential randomness in outputs



# Testing Individual Components and Ablations

- Log inputs and outputs to agents
- Create a benchmark using logged data
- Evaluate standalone components

# Human Validation

- Even if human validation is not part of the experimental method, you should sanity check your outputs!

# Section V: Transparency and Reproducibility

- Goal for this section: how to ensure LLM subcomponents are reproducible

- Overview
  - Reproducibility requires transparency
  - What to report
  - Logging LLM inputs/outputs
  - Anonymizing LLM artifacts for peer review

# Reproducibility and Transparency

- LLMs are stochastic
  - There are ways to make them reproducible, but it is difficult or impossible for large open models or commercial models
  - Thus, generated outputs need to be part of the artifact to make them reproducible
- An independent researcher should be able to reproduce your research by “replaying” generated outputs, instead of generating them again

# Logging Generations and Making Them Available for Peer Review

- APIs
  - Simply log API outputs or wrap API calls in a logging function
- Agents
  - Most, if not all, agent frameworks provide tracing functionality
  - Use agent traces to save sequence of actions and generations
- Anonymizing agent traces
  - Anonymizing requirements vary by venue
  - Tools like anonymous4science can host files and mask out provided key words – it provides a link that can be included in a paper or artifact submission

# Logging Agent Calls

- OpenAI Agents SDK
  - Traces are logged by default; OPENAI_AGENTS_DISABLE_TRACING=1
- LangChain
  - LANGSMITH_TRACING=true
- LlamaIndex
  - mlflow.llama_index.autolog() use MLFlow to log
- AutoGen
  - Uses Python logging

# Replaying from Logs Concept

```
client = OpenAI(http_client=httpx.Client(
                transport=RecordingTransport(cache_dir="…"))
)

# Transport intercepts every API request
class RecordingTransport(httpx.BaseTransport):
        def handle_request(self, request):
                key = hash_request(request)

                # Replay mode: return saved response
                if REPLAY:
                        return load_response_from_log(key)

                # Normal mode: call OpenAI
                response = real_transport.handle_request(request)
                body = response.read()

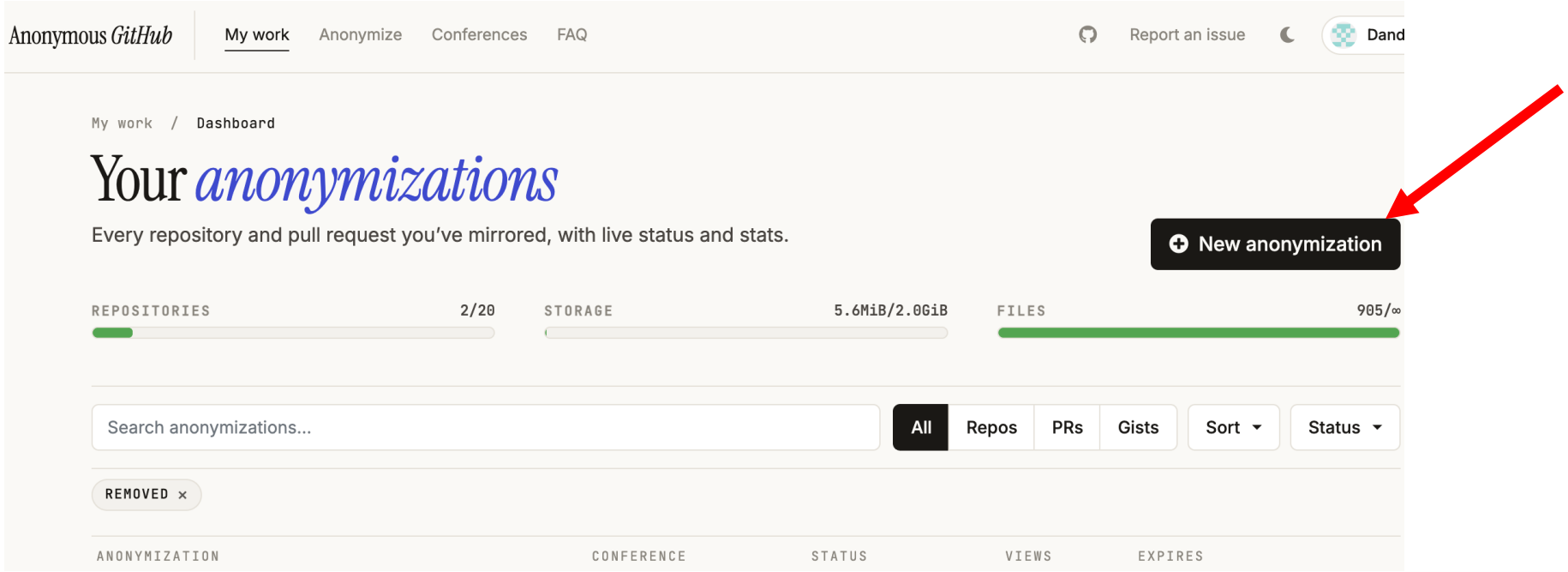
                # Log request + response
                save_json(key, response)
                return rebuild_response(response, body)
```

It only takes tens of lines of code to intercept API calls, log them, and/or replay them

See replaying API calls demo

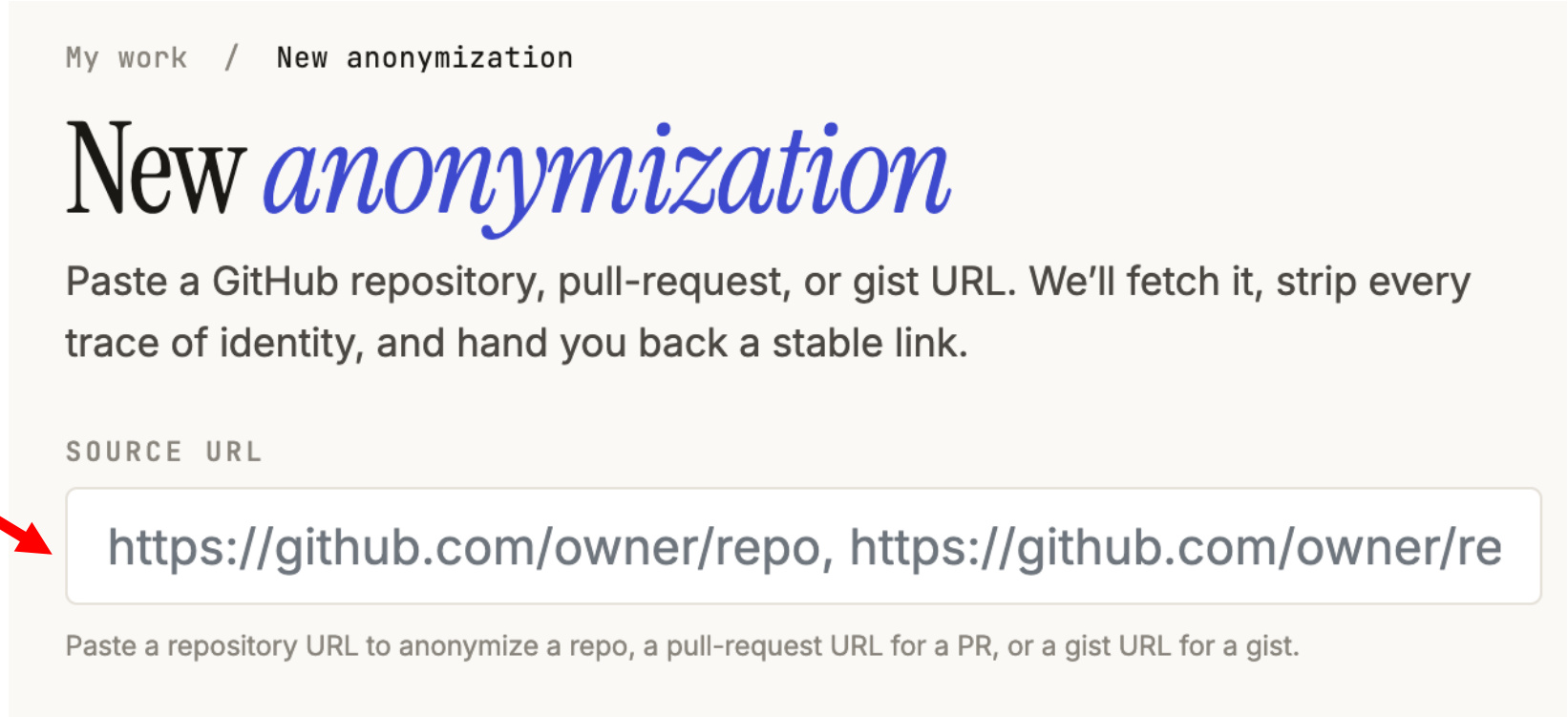
# Hosting Anonymous Data with Anonymous4science

- https://anonymous.4open.science/



# Hosting Anonymous Data with Anonymous4science

- https://anonymous.4open.science/



# Where to Report

# Section VI: Human Validation: When and How

- Goal for this section: how should humans interact with LLM research components

- Overview
  - When to keep humans in the loop
  - How to employ human evaluation on LLM subcomponents

# Issues with Human Validation

- Humans are inconsistent scorers
  - Even the same person may score things differently across time
- It is not reproducible
- Humans are very easily swayed; it is easy to measure the wrong metric
  - Example: people often find longer, more technical answers better regardless of their correctness

# When and How to Use Human Validation

- Use as a supplement to reproducible metrics
- All human scoring and raw results need to be publicly available
  - This is part of the methodology now
- Use metrics that are less susceptible to subjectivity
  - Example: rather than ask “Which output summarizes the scientific result better?”, instead ask “Based on the following generated output, please answer the following multiple-choice questions about a scientific result?”
  - Provide a reproducible metric

# Good Human Validation

Explaining Performance Profiles

# Section VII: Baselines, Metrics, and Ablations

- Goal for this section: best practices for experimental setups for LLM-based research

- Overview
  - What's a good baseline
  - Metrics for LLMs
  - How to structure ablation studies

# LLM Baselines

- Representative baselines are important!
- Compare with models of similar: cost, performance (time), quality, availability
- Compare with frontier models / agents
  - Being the best open model is valuable research
  - But comparison to SotA is still necessary context

# Probabilistic Metrics

- Since LLMs are random, we care about *how often* they are correct
- For every evaluation using an LLM we need to gather many samples to compute a probabilistic metric
- Common examples:
  - Harmonic and geometric means
  - metric@k e.g. pass@k

# @k Metrics For Binary Outputs

- For **binary outputs** @k style metrics measure the probability an LLM gets **at least** one particular output type **given *k* attempts**.
- Example
  - We are evaluating the rate at which LLMs write code with *race conditions*
  - For a given coding task, let *race@k* be the probability that, if the LLM has *k* attempts to write the code, at least one of the *k* will have a race condition
  - Generate $N \gg k$ outputs with the LLM

$$\text{race@k} = 1 - \binom{N-r}{k} \Big/ \binom{N}{k}$$

# Probabilistic Metrics

- pass@k – How often are outputs correct?

$$\text{pass@}k = \frac{1}{|P|} \sum_{p \in P} \left[ 1 - \binom{N - c_p}{k} \Big/ \binom{N}{k} \right]$$

*N* samples generated per prompt

Set of prompts

Number of correct generations

**prompt**

```
/* Compute the dot product of x and y using
OpenMP */
int product(int *x, int *y, size_t N) {
```

LLM

What is the probability at least one of k responses is correct?

**response 1**

```
int sum = 0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
  sum += x[i] * y[i];
}
return sum;
```

**response 2**

```
int sum = 0;
for (int i = 0; i < N; i++) {
  sum += x[i] * y[i];
}
return sum;
```

⋮

**response k**

# @k Metrics For Numeric Outputs

- Rather than measuring the probability of at least one correct, we instead measure the **expected best** over *k* attempts
- Example
  - We are evaluating the rate at which LLMs write code with *race conditions*
  - For a given coding task, let *race@k* be the probability that, if the LLM has *k* attempts to write the code, at least one of the *k* will have a race condition
  - Generate $N \gg k$ outputs with the LLM

$$\text{race@k} = 1 - \binom{N-r}{k} \Big/ \binom{N}{k}$$

# Measuring Performance of LLM Generated Code

- speedup@k – What's the expected max speedup?

$$\mathbb{E}\left[\max\left\{\frac{T^*}{T_{s_1}}, \ldots, \frac{T^*}{T_{s_k}}\right\}\right]$$

Baseline runtime

Runtime of LLM generations

**prompt**

```
/* Compute the dot product of x and y
using OpenMP */
int product(int *x, int *y, size_t N) {
```

LLM

What is the *expected max speedup* over k response?

**response 1**

```
int sum = 0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
   sum += x[i] * y[i];
}
return sum;
```

**response 2**

```
int sum = 0;
for (int i = 0; i < N; i++) {
   sum += x[i] * y[i];
}
return sum;
```

⋮

**response k**

# Measuring Performance of LLM Generated Code

- speedup@k – What's the expected max speedup?

Baseline sequential runtime

$$\mathbb{E}\left[\max\left\{\frac{T^*}{T_{s_1,n}},\ldots,\frac{T^*}{T_{s_k,n}}\right\}\right]$$

Runtime of LLM generations on *n* processors

**prompt**

```
/* Compute the dot product of x and y
using OpenMP */
int product(int *x, int *y, size_t N) {
```

LLM

What is the *expected max speedup* over k response?

**response 1**

```
int sum = 0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
   sum += x[i] * y[i];
}
return sum;
```

**response 2**

```
int sum = 0;
for (int i = 0; i < N; i++) {
   sum += x[i] * y[i];
}
return sum;
```

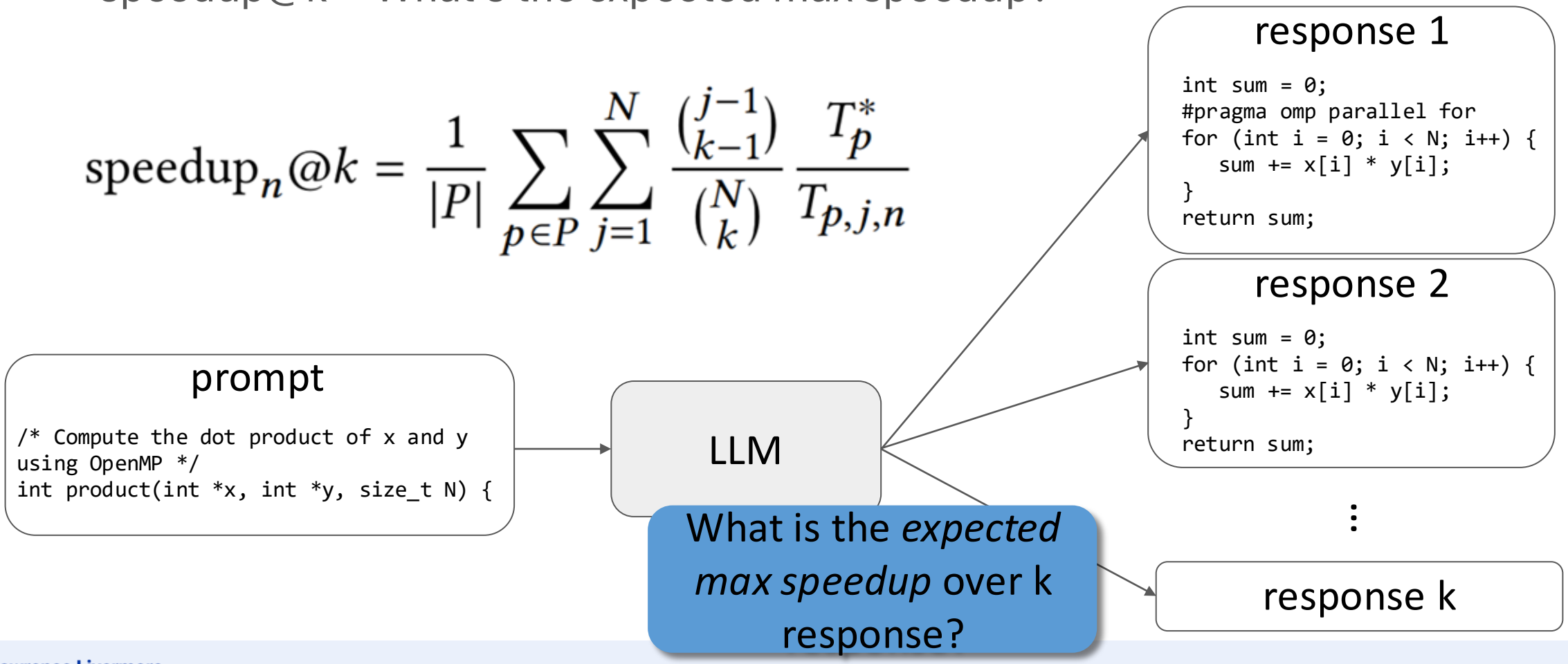
⋮

**response k**

# Measuring Performance of LLM Generated Code

- speedup@k – What's the expected max speedup?

$$\text{speedup}_n@k = \frac{1}{|P|} \sum_{p \in P} \sum_{j=1}^{N} \frac{\binom{j-1}{k-1}}{\binom{N}{k}} \frac{T_p^*}{T_{p,j,n}}$$



# Conclusion